KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

VEHICLE SERVICE REMINDER AND MAINTENANCE TRACKER

Submitted by

SHABIKSHA.B

25BCS148

Under the Guidance of

MR.ARAVIND

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **2**

DECLARATION

I hereby declare that the project entitled "**Vehicle Service Reminder and Maintenance Tracker"** submitted in partial fulfillment of the requirements for the course **JAVA PROGRAMMING** is a **record of my original work** carried out under the guidance of **MR.ARAVIND**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

SHABIKSHA.B

CERTIFICATE

This is to certify that the project entitled **"Vehicle Service Reminder and Maintenance Tracker"** is a **bona fide work** carried out by **[SHABIKSHA B] [CS-1'B'-2025]**, **[COMPUTER SCIENCE]**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[JAVA PROGRAMMING]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: COIMBATORE

Date:10-04-2026

Submitted for the Viva-Voce Examination held on 10-04-2026

Guide:
(Technical Trainer, KGCAS)

Mr.Aravind

Head of the Department
(Technical Head,KGCAS)

Dr. Boopalan Sundaramoorthy

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	**6**
1.1	Objective	6
1.2	Overview	6
1.3	Features	7
2	**Program Analysis**	8
2.1	Problem Definition	8
2.2	Project Overview	8
2.3	Scopes and Limitations	8
3	**System Design**	**9**
3.1	Modules	9
4	**Methodology**	**10**
4.1	Problem Analysis	10
4.2	System Design	10
4.3	Program Development	11
4.4	Testing and Validation	11
4.5	Result and Documentation	12
4.6	Algorithm	12
4.7	Flowchart	13
5	**Source Code**	**15**
6	**Input and Output Screens**	**22**
7	**Discussion and Conclusion**	**23**
7.1	Discussion	23
7.2	Conclusion	23
8	**References**	**24**

ABSTRACT

The **Vehicle Service Reminder and Maintenance Tracker** is a Java-based application developed to help users manage vehicle maintenance schedules effectively. It allows users to enter vehicle details such as name, last service date, and service interval through a simple graphical user interface. The system is designed to ensure that users do not miss important service deadlines.The application processes the input data using the Java Time API to calculate the next service date. Based on this calculation, it provides reminders indicating whether the service is due, due today, or overdue. This helps users take timely action and maintain their vehicles properly.In addition to reminders, the system stores service records using file handling techniques. Users can view the stored data in a table format and search for specific vehicle records easily. This improves organization and makes it convenient to access past maintenance history. Overall, the project demonstrates the use of Java concepts such as GUI development, event handling, file management, and date processing. It provides a simple, user-friendly solution for vehicle service tracking and can be further enhanced with advanced features in the future.

1. INTRODUCTION

In today's fast-paced world, managing regular vehicle maintenance has become a challenge for many users. People often forget important service dates due to busy schedules, which can lead to poor vehicle performance and increased repair costs. Proper maintenance is essential to ensure the safety, efficiency, and longevity of any vehicle.

With the advancement of technology, software applications can simplify such daily tasks by providing timely reminders and organized record-keeping. A digital system is more reliable than manual tracking, as it reduces human error and ensures that important dates are not missed.

The **Vehicle Service Reminder and Maintenance Tracker** is developed to address this problem by offering a simple and effective solution. It helps users track service schedules, receive reminders, and maintain records of vehicle maintenance in an organized manner.

This project is implemented using Java, utilizing concepts such as GUI design, event handling, file management, and date processing. It aims to provide a user-friendly interface that makes vehicle maintenance tracking easy, efficient, and accessible.

1.1 Objective

The main objective of the **Vehicle Service Reminder and Maintenance Tracker** is to develop a simple and efficient system to manage vehicle service schedules. It aims to provide timely reminders to users so that they do not miss important maintenance activities, ensuring better vehicle performance and safety.

Another objective of this project is to store and organize service records in a structured manner, allowing users to easily view and search their vehicle history. The system is also designed to offer a user-friendly interface using Java Swing, while applying important programming concepts such as file handling and date processing.

1.2 Overview

The **Vehicle Service Reminder and Maintenance Tracker** is a simple Java-based application designed to help users manage and monitor vehicle maintenance schedules. It allows users to enter vehicle details, calculate upcoming service dates, and receive reminders to ensure timely servicing.

The system also provides features to store service records, display them in a table format, and search for specific entries. With a user-friendly interface and basic file handling, the application offers an efficient and organized way to keep track of vehicle maintenance history.

1.3 Features

Allows users to enter vehicle details and service information

- Calculates next service date automatically
- Provides reminders (Due / Today / Overdue)
- Saves service records using file handling
- Displays records in table format
- Enables search of vehicle records
- User-friendly graphical interface using Java Swing
- Simple and easy to use system

2. PROGRAM ANALYSIS

The **Vehicle Service Reminder and Maintenance Tracker** program is designed to process user input and provide accurate service reminders. It takes vehicle details such as name, last service date, and service interval as input. The program then uses the Java Time API to calculate the next service date and compares it with the current date to determine the service status.

Based on the comparison, the program displays appropriate messages like due, due today, or overdue. It also uses file handling to store and retrieve service records. Additional features like displaying records in a table and searching for specific vehicles improve the usability of the program. Overall, the program efficiently handles input, processing, and output operations to provide reliable results.

2.1 Problem Definition

Many vehicle owners face difficulty in remembering their vehicle service schedules, which can lead to missed maintenance and reduced vehicle performance. There is no proper system to provide timely reminders or to maintain service records in an organized way.

Hence, there is a need to develop a simple application that can track vehicle service dates, provide reminders, and store maintenance history efficiently.

2.2 Project Overview

The **Vehicle Service Reminder and Maintenance Tracker** is a Java-based desktop application designed to help users manage vehicle maintenance efficiently. It allows users to enter vehicle details, track service schedules, and receive timely reminders for upcoming or overdue services.

The system also provides features to store service records, display them in a table format, and search for specific vehicle details. Developed using Java Swing and file handling techniques, the project offers a simple, user-friendly solution for maintaining vehicle service history and ensuring proper vehicle care.

2.3 Scopes and Limitations

Scope

- Provides timely reminders for maintenance
- Stores and manages service history records
- Allows viewing and searching of records
- Can be enhanced with database and mobile features in future
- Helps users track vehicle service schedules easily

3. SYSTEM DESIGN

The system is designed as a Java-based desktop application that follows a simple Input → Process → Output structure. The user enters vehicle details such as name, last service date, and service interval through a graphical user interface developed using Java Swing. The system processes this data using the Java Time API to calculate the next service date and determine the service status.

The output is displayed as reminder messages indicating whether the service is due, due today, or overdue. The system also uses file handling to store vehicle records, which can be viewed in a table format using JTable. A search feature is included to easily find specific records, ensuring the system is simple, efficient, and user-friendly.

3.1 Modules

The main modules of the **Vehicle Service Reminder and Maintenance Tracker** are explained below:

Input Module – Collects vehicle name, service date, and interval from the user

Processing Module – Calculates next service date using input data

Reminder Module – Displays service status (Due / Today / Overdue)

Storage Module – Saves vehicle records into a file

Display Module – Shows saved records in table format

Search Module – Allows user to search records by vehicle name

4. METHODOLOGY

The development of the **Vehicle Service Reminder and Maintenance Tracker** followed a systematic approach. Initially, the problem was analyzed and the requirements were identified. A simple design was created based on the input, process, and output model. The application was then developed using Java in the Eclipse IDE, with Java Swing used to design the graphical user interface.

The system accepts user input such as vehicle details, last service date, and service interval. It processes this data using the Java Time API to calculate the next service date and generate appropriate reminders. File handling techniques were used to store and retrieve service records. Finally, the system was tested with different inputs to ensure accurate results and smooth performance.

4.1. Problem Analysis

In today's busy lifestyle, vehicle owners often forget important service dates, which can lead to poor vehicle performance and increased maintenance issues. Managing service schedules manually is difficult and unreliable, as there is no proper system to remind users at the right time.

Additionally, maintaining records of past services and searching for them when needed becomes inconvenient. Therefore, there is a need for a simple and user-friendly system that can provide timely reminders, store maintenance records, and allow easy access to vehicle service history efficiently.

4.2. System Design

The system is designed as a **desktop application** using Java

- It follows a simple **Input → Process → Output** model
- **Input Module:** Accepts vehicle name, service date, and interval
- **Processing Module:** Calculates next service date using Java Time API
- **Reminder Module:** Compares dates and shows service status
- **Storage Module:** Saves records in a text file
- **Display Module:** saved records in a table format

- **Search Module:** Filters records based on vehiCLE
- **User Interface:** Designed using Java Swing with buttons and text fields
- The system ensures **easy interaction and accurate results**

4.3. Program Development

Identified the problem and requirements of the system

- Designed basic flow (input, process, output)
- Created project in Eclipse using Java
- Developed GUI using Java Swing
- Added input fields for vehicle details
- Implemented date calculation using Java Time API
- Created reminder logic (Due / Today / Overdue)
- Added file handling to save records
- Implemented history display using JTable
- Added search functionality for vehicles
- Tested the program and fixed errors

4.4. Testing and Validation

The **Vehicle Service Reminder and Maintenance Tracker** was tested using different input values to ensure accuracy and reliability. Various test cases were used, including valid inputs, invalid inputs, and boundary conditions.

- **Valid Input Testing:**
  The system correctly calculated the next service date and displayed appropriate reminders (due, due today, overdue).

- **Invalid Input Testing:**
  When incorrect data (like wrong date format or empty fields) was entered, the system displayed error messages, ensuring proper validation.

- **File Handling Testing:**
  Records were successfully saved and retrieved from the file without data loss.

- **Search and Display Testing:**
  The history table displayed all records correctly, and the search feature accurately filtered results based on vehicle name.

The results confirm that the system performs as expected and handles different scenarios effectively, ensuring dependable performance.

4.5. Result and Documentation

The **Vehicle Service Reminder and Maintenance Tracker** was successfully implemented using Java. The system accepts vehicle details, calculates the next service date, and provides accurate reminders such as due, due today, or overdue.

It also allows users to save service records, view them in a table format, and search for specific vehicles. The program executed without errors and produced the expected results, demonstrating its effectiveness in managing vehicle maintenance.

4.6. Algorithm

Algorithm for the **Vehicle Service Reminder and Maintenance Tracker**:

1. Start

- Enter vehicle name, last service date, and service interval
- Convert input into date format
- Calculate next service date
- Get current date
- Compare current date with next service date

- Display reminder (Due / Today / OVER DUE)
- If required, save record to file
- If requested, display or search history
- End

4.7. Flowchart

START

↓

Enter Vehicle Details

(Name, Last Service Date, Interval)

↓

Convert Input to Date Format

↓

Calculate Next Service Date

↓

Get Current Date

↓

Compare Dates

↓

Is Service Due?

↓

┌──────────────┬──────────────┬──────────────┐

↓ ↓ ↓

Days > 0 Days = 0 Days < 0

↓ ↓ ↓

Show "Due in X days" Show "Due Today" Show "Overdue by X days"

↓

Ask User Action

(Check / Save / View / Search)

↓

If Save → Store in File

↓

If View → Display Table

↓

If Search → Filter Records

↓

END

5. SOURCE CODE

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;

public class VehicleServiceGUI extends JFrame implements ActionListener {

    JTextField vehicleField, dateField, intervalField, searchField;

JButton checkButton, saveButton, viewButton, searchButton;

    public VehicleServiceGUI() {

        setTitle("Vehicle Service Reminder");
        setSize(550, 420);
        setLayout(null);

        JPanel panel = new JPanel();
        panel.setLayout(null);
        panel.setBackground(new Color(245, 250, 255));
        panel.setBounds(0, 0, 550, 420);
        add(panel);
```

```
JLabel title = new JLabel(" 🚚 Service Tracker");
        title.setBounds(170, 10, 250, 30);
        title.setFont(new Font("Segoe UI", Font.BOLD, 20));
        title.setForeground(new Color(60, 90, 150));
        panel.add(title);

        JLabel vLabel = new JLabel("Vehicle Name:");
        vLabel.setBounds(40, 70, 120, 25);
        panel.add(vLabel);

        vehicleField = new JTextField();
        vehicleField.setBounds(180, 70, 200, 28);
        panel.add(vehicleField);

        JLabel dLabel = new JLabel("Last Service Date:");
        dLabel.setBounds(40, 110, 140, 25);
        panel.add(dLabel);

        dateField = new JTextField("dd-MM-yyyy");
        dateField.setBounds(180, 110, 200, 28);
        panel.add(dateField);

        JLabel iLabel = new JLabel("Interval (days):");
        iLabel.setBounds(40, 150, 120, 25);
        panel.add(iLabel);

        intervalField = new JTextField();
        intervalField.setBounds(180, 150, 200, 28);
        panel.add(intervalField);
```

```
    checkButton = new JButton(" 🔔 Check");
    checkButton.setBounds(40, 200, 120, 35);
    checkButton.setBackground(new Color(120, 180, 255));
    checkButton.setForeground(Color.WHITE);
    panel.add(checkButton);

    saveButton = new JButton(" 💾 Save");
    saveButton.setBounds(180, 200, 120, 35);
    saveButton.setBackground(new Color(100, 200, 150));
    saveButton.setForeground(Color.WHITE);
    panel.add(saveButton);

    viewButton = new JButton(" 📊 History”)
viewButton.setBounds(320, 200, 150, 35);
    viewButton.setBackground(new Color(180, 140, 255));
    viewButton.setForeground(Color.WHITE);
    panel.add(viewButton);

    // Search section
    JLabel searchLabel = new JLabel("Search Vehicle:");
    searchLabel.setBounds(40, 270, 120, 25);
    panel.add(searchLabel);
     searchField = new JTextField();
    searchField.setBounds(180, 270, 200, 28);
    panel.add(searchField);
  searchButton = new JButton(" 🔍 Search");
    searchButton.setBounds(180, 310, 150, 35);
    searchButton.setBackground(new Color(255, 170, 120));
    searchButton.setForeground(Color.WHITE);
    panel.add(searchButton);
```

```
        checkButton.addActionListener(this);
        saveButton.addActionListener(this);
        viewButton.addActionListener(this);
        searchButton.addActionListener(this);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }
    public void actionPerformed(ActionEvent e) {

        if (e.getSource() == viewButton) {
            showTable(null);
            return;
        }

        if (e.getSource() == searchButton) {
            String keyword = searchField.getText().toLowerCase();
            showTable(keyword);
            return;
        }

        String vehicle = vehicleField.getText();
        String dateInput = dateField.getText();
        String intervalText = intervalField.getText();
try {
            int interval = Integer.parseInt(intervalText);
            DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy");
            LocalDate lastDate = LocalDate.parse(dateInput, formatter);
            LocalDate nextDate = lastDate.plusDays(interval);
            LocalDate today = LocalDate.now();
```

```
            long days = ChronoUnit.DAYS.between(today, nextDate);

            String message;
if (days > 0) {
                message = "Due in " + days + " days";
            } else if (days == 0) {
                message = "Due TODAY";
            } else {
                message = "OVERDUE by " + Math.abs(days) + " days";
            }

            JOptionPane.showMessageDialog(this,
                    " 🚙 " + vehicle +
                    "\nNext Service: " + nextDate +
                    "\n" + message);

            if (e.getSource() == saveButton) {
                FileWriter fw = new FileWriter("service_records.txt", true);
                fw.write(vehicle + "," + lastDate + "," + nextDate + "," + message + "\n");
                fw.close();

                JOptionPane.showMessageDialog(this, " ✅ Record Saved!");
            }

        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "⚠ Invalid Input!");
        }
    }
```

```
// SHOW TABLE (WITH OPTIONAL SEARCH)
    public void showTable(String keyword) {
      JFrame frame = new JFrame(" 📊 Service History");
        frame.setSize(650, 300);

        String[] columns = {"Vehicle", "Last Service", "Next Service", "Status"};
        DefaultTableModel model = new DefaultTableModel(columns, 0);

        try {
            BufferedReader br = new BufferedReader(new FileReader("service_records.txt"));
            String line;

            while ((line = br.readLine()) != null) {
                String[] data = line.split(",");

                if (keyword == null || data[0].toLowerCase().contains(keyword)) {
                    model.addRow(data);
                }
            }

            br.close();

        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "No records found!");
        }

         JTable table = new JTable(model);
        table.setRowHeight(25);
```

```
    JScrollPane scrollPane = new JScrollPane(table);
        frame.add(scrollPane);

        frame.setVisible(true);
    }

    public static void main(String[] args) {
        new VehicleServiceGUI();
    }
}
```

6. INPUT AND OUTPUT SCREENS

Figure 1: Menu

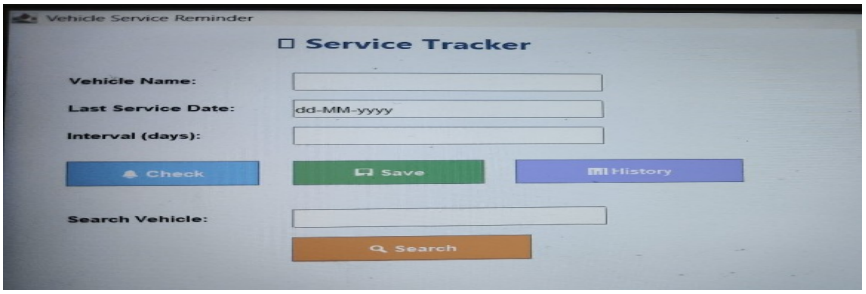
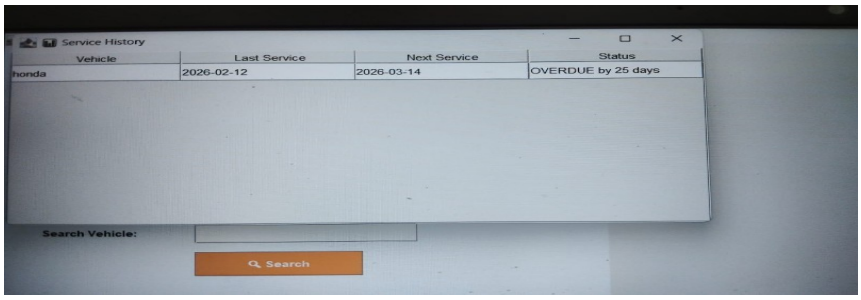


Figure 2: HISTORY Output



7. DISCUSSION & CONCLUSION

7.1 Discussion

The development of the **Vehicle Service Reminder and Maintenance Tracker** highlights how simple software solutions can effectively solve everyday problems. Through this project, important concepts of Java such as Swing-based GUI design, event handling, file storage, and date manipulation were applied in a practical way.

One of the key strengths of the system is its ability to provide timely reminders and maintain service history, which improves user convenience and reduces the risk of missing important maintenance schedules. The inclusion of features like history display and search functionality makes the system more interactive and user-friendly.

However, the current system has some limitations, such as the use of basic file handling instead of a database and the absence of real-time notifications. These limitations open opportunities for future improvements like integrating a database, adding mobile alerts, and enhancing the user interface.

Overall, the project demonstrates how programming concepts can be transformed into a useful real-world application, making vehicle maintenance tracking simpler and more efficient.

7.2 Conclusion

The **Vehicle Service Reminder and Maintenance Tracker** was developed using Java to help users manage vehicle service schedules efficiently. It provides timely reminders, stores maintenance records, and allows easy viewing and searching of past data through a user-friendly interface.

This project demonstrates key Java concepts like GUI design, event handling, file management, and date processing. Overall, it serves as a simple and reliable system to ensure timely vehicle maintenance and better organization.

8. REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com